Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Трекинг множественных объектов»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Сугак В.А.
Проверил:
Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов

Общее задание

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку **U**ltralytics YOLO;

2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);

3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;

4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Ход работы

Код программы:

```
import os
import shutil
import subprocess
import time
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import pandas as pd
import ultralytics
import yaml
from ultralytics import YOLO

WEIGHTS = "/content/drive/MyDrive/lab3_yolo26m/weights/best.pt"
VIDEO = "/content/video_crop.mp4" if os.path.exists("/content/video_crop.mp4") \
    else "/content/video_fixed.mp4"
CONF, IMGSZ, CLASSES = 0.03, 1280, None
BASE_LOW = {"track_high_thresh": 0.05, "track_low_thresh": 0.03, "new_track_thresh": 0.05}
SAVE_VIDEO_FOR = ["bytetrack_default", "botsort_default"]

OUT = Path("/content/tracking_results")
(OUT / "configs").mkdir(parents=True, exist_ok=True)
CFG_DIR = Path(ultralytics.__file__).parent / "cfg" / "trackers"

assert cv2.VideoCapture(VIDEO).isOpened(), "Видео не открывается, проверь путь"
cap = cv2.VideoCapture(VIDEO)
N_FRAMES = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print("Видео:", VIDEO, "| кадров:", N_FRAMES, "|", int(cap.get(3)), "x", int(cap.get(4)))
cap.release()

best = YOLO(WEIGHTS)
print("Классы модели:", best.names)

cap = cv2.VideoCapture(VIDEO)
fig, axes = plt.subplots(1, 4, figsize=(20, 7))
for ax, idx in zip(axes, [0, N_FRAMES // 3, 2 * N_FRAMES // 3, N_FRAMES - 2]):
    cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
    ok, frame = cap.read()
    if not ok:
        continue
    res = best.predict(frame, conf=0.01, imgsz=IMGSZ, verbose=False)[0]
    confs = [round(c, 2) for c in res.boxes.conf.tolist()]
    print("кадр", idx, "| найдено:", len(confs), "| уверенность:", sorted(confs, reverse=True)[:8])
    ax.imshow(res.plot()[..., ::-1])
    ax.axis("off")
    ax.set_title(f"кадр {idx}: найдено {len(confs)}")
cap.release()
plt.tight_layout()
plt.savefig(OUT / "detection_frames.png", dpi=130)
```

```
plt.show()

EXPERIMENTS = [
    ("bytetrack_default", "bytetrack", {}),
    ("botsort_default", "botsort", {}),
    ("bytetrack_buffer15", "bytetrack", {"track_buffer": 15}),
    ("bytetrack_buffer60", "bytetrack", {"track_buffer": 60}),
    ("bytetrack_buffer120", "bytetrack", {"track_buffer": 120}),
    ("bytetrack_high0.15", "bytetrack", {"track_high_thresh": 0.15, "new_track_thresh": 0.15}),
    ("bytetrack_high0.3", "bytetrack", {"track_high_thresh": 0.3, "new_track_thresh": 0.3}),
    ("bytetrack_match0.6", "bytetrack", {"match_thresh": 0.6}),
    ("bytetrack_match0.95", "bytetrack", {"match_thresh": 0.95}),
    ("botsort_buffer60", "botsort", {"track_buffer": 60}),
    ("botsort_high0.15", "botsort", {"track_high_thresh": 0.15, "new_track_thresh": 0.15}),
    ("botsort_gmc_none", "botsort", {"gmc_method": "none"}),
    ("botsort_prox0.3", "botsort", {"proximity_thresh": 0.3}),
    ("botsort_prox0.7", "botsort", {"proximity_thresh": 0.7}),
    ("botsort_reid", "botsort", {"with_reid": True}),
]

def compute_metrics(frames_ids, short_len=15):
    tracks = {}
    for f, ids in enumerate(frames_ids):
        for i in ids:
            tracks.setdefault(i, []).append(f)
    lengths = [len(v) for v in tracks.values()]
    gaps = sum(sum(1 for a, b in zip(v, v[1:]) if b - a > 1) for v in tracks.values())
    per_frame = [len(x) for x in frames_ids]
    n = len(frames_ids)
    short = sum(l < short_len for l in lengths)
    metrics = {
        "ID": len(tracks),
        "Объектов на кадр": round(sum(per_frame) / n, 2) if n else 0,
        "Макс. одновременно": max(per_frame) if per_frame else 0,
        "Средняя длина": round(sum(lengths) / len(lengths), 1) if lengths else 0,
        "Медиана": sorted(lengths)[len(lengths) // 2] if lengths else 0,
        "Коротких, %": round(100 * short / len(lengths), 1) if lengths else 0,
        "Gaps": gaps,
        "Кадров без объектов": sum(x == 0 for x in per_frame),
    }
    return metrics, tracks, per_frame, lengths

def plot_timeline(tracks, n_frames, title, path):
    fig, ax = plt.subplots(figsize=(10, max(3, 0.25 * len(tracks) + 1)))
    for row, (tid, fr) in enumerate(sorted(tracks.items(), key=lambda kv: kv[1][0])):
        ax.plot(fr, [row] * len(fr), ".", markersize=2)
    ax.set_xlim(0, max(n_frames, 1))
    ax.set_xlabel("кадр")
    ax.set_ylabel("трек")
    ax.set_title(title)
    plt.tight_layout()
    fig.savefig(path, dpi=130)
    plt.close(fig)

rows, per_frame_all, lengths_all = [], {}, {}
for name, base, overrides in EXPERIMENTS:
    print("===", name, overrides)
    cfg = yaml.safe_load((CFG_DIR / f"{base}.yaml").read_text())
    cfg.update(BASE_LOW)
    cfg.update(overrides)
    cfg_path = OUT / "configs" / f"{name}.yaml"
    cfg_path.write_text(yaml.safe_dump(cfg, sort_keys=False))
    try:
        cap = cv2.VideoCapture(VIDEO)
        fps = cap.get(cv2.CAP_PROP_FPS) or 25
        w, h = int(cap.get(3)), int(cap.get(4))
        cap.release()
        writer = None
        if name in SAVE_VIDEO_FOR:
```

```
            raw = OUT / f"{name}_raw.mp4"
            writer = cv2.VideoWriter(str(raw), cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))
        frames_ids, t0 = [], time.time()
        for r in YOLO(WEIGHTS).track(source=VIDEO, tracker=str(cfg_path), conf=CONF,
                                     imgsz=IMGSZ, classes=CLASSES, stream=True,
                                     persist=True, verbose=False):
            frames_ids.append([] if r.boxes.id is None else [int(x) for x in r.boxes.id.tolist()])
            if writer is not None:
                writer.write(r.plot())
        dt = time.time() - t0
        if writer is not None:
            writer.release()
            subprocess.run(["ffmpeg", "-y", "-i", str(raw), "-vcodec", "libx264",
                            "-pix_fmt", "yuv420p", str(OUT / f"{name}.mp4"),
                            "-loglevel", "quiet"])
        m, tracks, pf, L = compute_metrics(frames_ids)
        m["FPS"] = round(len(frames_ids) / dt, 1)
        plot_timeline(tracks, len(frames_ids), name, OUT / f"{name}_timeline.png")
        per_frame_all[name], lengths_all[name] = pf, L
        rows.append({"Эксперимент": name, "Алгоритм": base,
                     "Изменение": str(overrides) if overrides else "по умолчанию", **m})
        print(m)
    except Exception as e:
        print("ошибка в", name, ":", e)

if not rows:
    raise SystemExit("Ни один эксперимент не прошёл, смотри ошибки выше")

df = pd.DataFrame(rows)
df.to_csv(OUT / "summary.csv", index=False, encoding="utf-8-sig")

cmp_cols = ["ID", "Макс. одновременно", "Объектов на кадр", "Средняя длина", "Медиана",
            "Коротких, %", "Gaps", "Кадров без объектов", "FPS"]
cmp_df = df[df["Эксперимент"].isin(["bytetrack_default", "botsort_default"])] \
    .set_index("Алгоритм")[cmp_cols].T
cmp_df.to_csv(OUT / "compare_default.csv", encoding="utf-8-sig")
print("\nСравнение ByteTrack и BoT-SORT (параметры по умолчанию):")
print(cmp_df)

tbl_cols = ["Эксперимент", "Изменение", "ID", "Объектов на кадр", "Средняя длина",
            "Медиана", "Коротких, %", "Gaps", "FPS"]
fig, ax = plt.subplots(figsize=(15, 0.45 * len(df) + 1.5))
ax.axis("off")
tbl = ax.table(cellText=df[tbl_cols].values, colLabels=tbl_cols, loc="center", cellLoc="center")
tbl.auto_set_font_size(False)
tbl.set_fontsize(9)
tbl.scale(1, 1.4)
plt.tight_layout()
fig.savefig(OUT / "table_experiments.png", dpi=150)
plt.show()

names = df["Эксперимент"].tolist()
colors = ["tab:blue" if a == "bytetrack" else "tab:orange" for a in df["Алгоритм"]]

fig, axs = plt.subplots(1, 4, figsize=(22, 6))
for ax, key, title in zip(axs, ["ID", "Средняя длина", "Коротких, %", "FPS"],
                          ["Уникальных ID (меньше = стабильнее)", "Средняя длина трека, кадров",
                           "Коротких треков, %", "Скорость, кадр/с"]):
    ax.barh(names, df[key], color=colors)
    ax.set_title(title)
    ax.invert_yaxis()
plt.tight_layout()
fig.savefig(OUT / "summary.png", dpi=130)
plt.show()

if "bytetrack_default" in per_frame_all and "botsort_default" in per_frame_all:
    plt.figure(figsize=(12, 4))
    plt.plot(per_frame_all["bytetrack_default"], label="ByteTrack")
    plt.plot(per_frame_all["botsort_default"], label="BoT-SORT")
```

```
        plt.xlabel("Номер кадра")
        plt.ylabel("Объектов на кадре")
        plt.title("Число отслеживаемых объектов по кадрам")
        plt.legend()
        plt.grid(alpha=0.3)
        plt.tight_layout()
        plt.savefig(OUT / "objects_per_frame.png", dpi=130)
        plt.show()

        plt.figure(figsize=(10, 4))
        plt.hist(lengths_all["bytetrack_default"], bins=20, alpha=0.6, label="ByteTrack")
        plt.hist(lengths_all["botsort_default"], bins=20, alpha=0.6, label="BoT-SORT")
        plt.xlabel("Длина трека, кадров")
        plt.ylabel("Число треков")
        plt.title("Распределение длины треков")
        plt.legend()
        plt.grid(alpha=0.3)
        plt.tight_layout()
        plt.savefig(OUT / "track_lengths.png", dpi=130)
        plt.show()

    fig, axs = plt.subplots(1, 2, figsize=(14, 6))
    axs[0].barh(names, df["Gaps"], color=colors)
    axs[0].set_title("Разрывов треков (gaps)")
    axs[0].invert_yaxis()
    axs[1].barh(names, df["Кадров без объектов"], color=colors)
    axs[1].set_title("Кадров без объектов")
    axs[1].invert_yaxis()
    plt.tight_layout()
    fig.savefig(OUT / "gaps.png", dpi=130)
    plt.show()
    for name, title in [("bytetrack_default", "ByteTrack"), ("botsort_default", "BoT-SORT")]:
        p = OUT / f"{name}.mp4"
        if not p.exists():
            continue
        cap = cv2.VideoCapture(str(p))
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        fig, axes = plt.subplots(1, 4, figsize=(20, 6))
        for ax, idx in zip(axes, [0, n // 3, 2 * n // 3, max(n - 2, 0)]):
            cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
            ok, fr = cap.read()
            if ok:
                ax.imshow(fr[..., ::-1])
                ax.set_title(f"{title}, кадр {idx}")
            ax.axis("off")
        cap.release()
        plt.tight_layout()
        fig.savefig(OUT / f"frames_{name}.png", dpi=130)
        plt.show()

    shutil.copytree(OUT, "/content/drive/MyDrive/lab4_tracking", dirs_exist_ok=True)
    print("Сохранено на Drive: lab4_tracking")
    df
```

Вывод программмы:

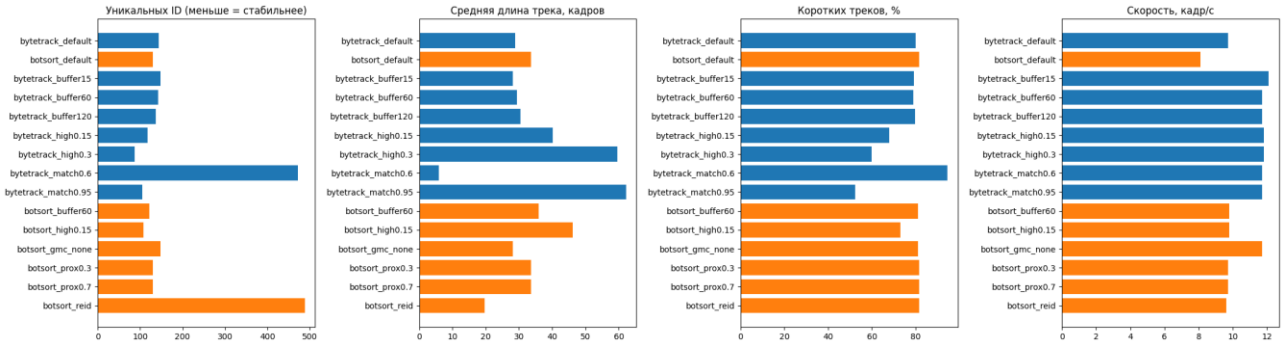
```
Видео: /content/video_fixed.mp4 | кадров: 1201 | 2088 x 1170
Классы модели: {0: '0', 1: '1', 2: '2', 3: '3', 4: '4', 5: '5', 6: '6', 7: '7', 8: '8', 9: '9', 10: 'counter', 11:
'liters'}
кадр 0 | найдено: 37 | уверенность: [0.73, 0.52, 0.5, 0.46, 0.44, 0.41, 0.34, 0.33]
кадр 400 | найдено: 7 | уверенность: [0.79, 0.47, 0.4, 0.23, 0.22, 0.02, 0.01]
кадр 800 | найдено: 11 | уверенность: [0.43, 0.38, 0.26, 0.06, 0.04, 0.02, 0.01, 0.01]
кадр 1199 | найдено: 9 | уверенность: [0.69, 0.41, 0.3, 0.04, 0.02, 0.01, 0.01, 0.01]
```

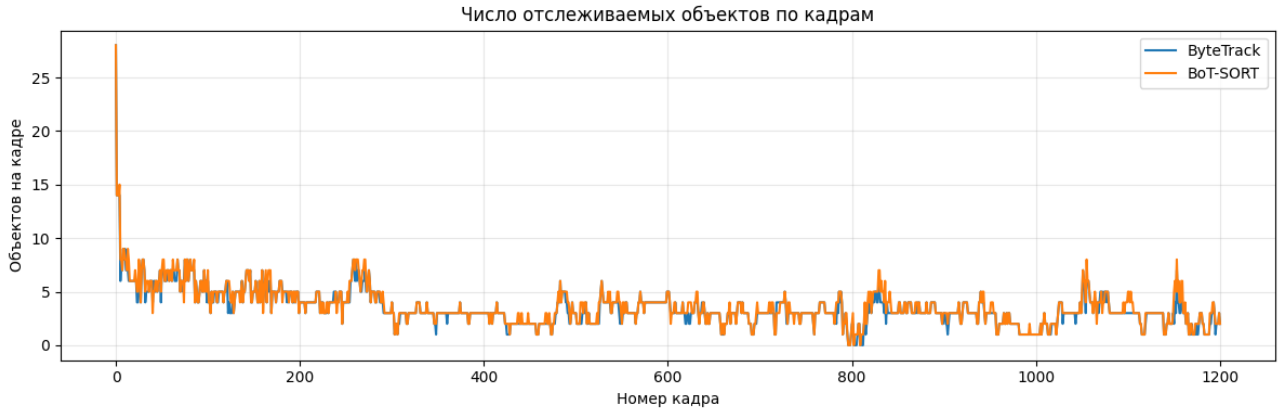


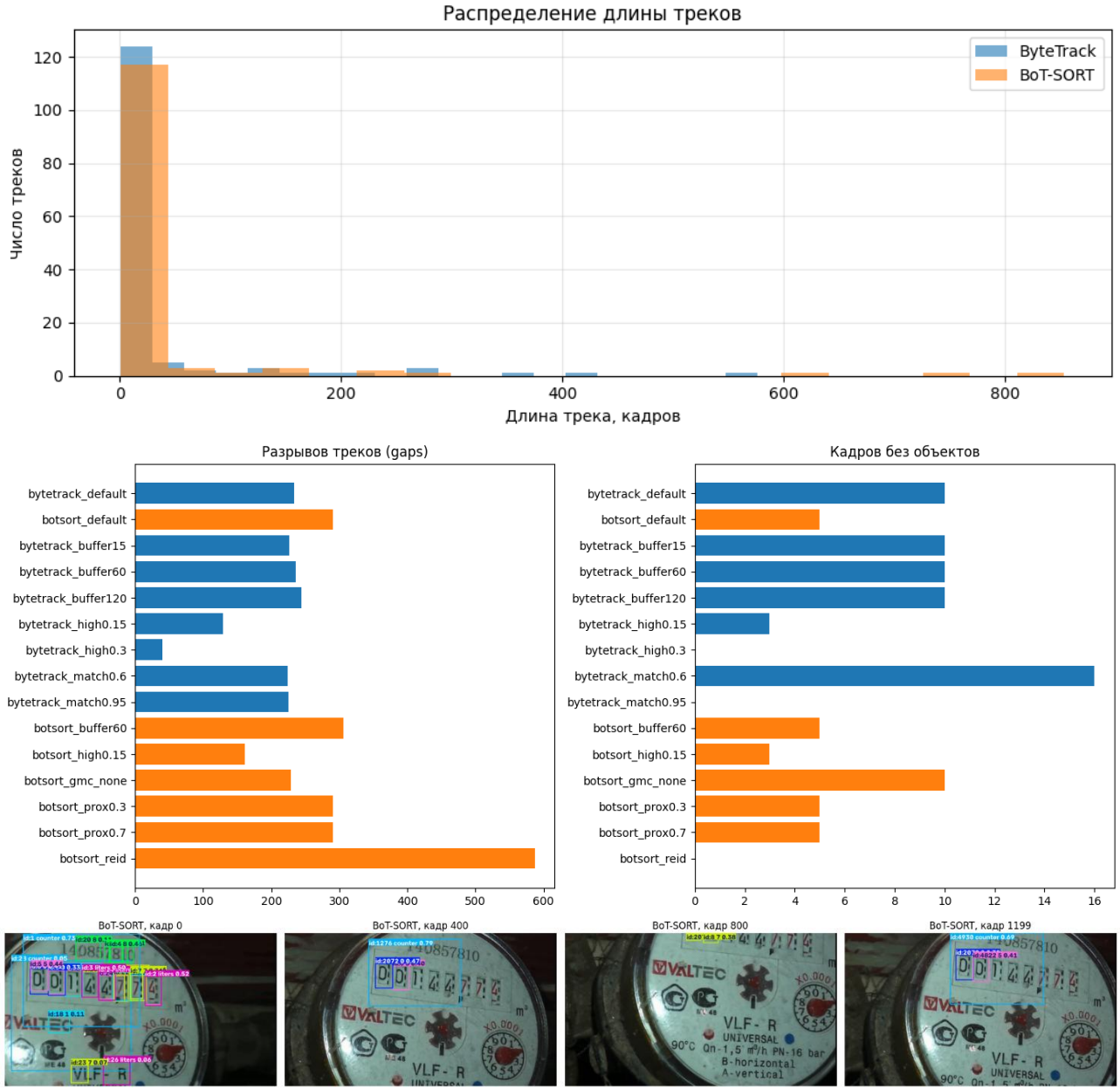
Сравнение ByteTrack и BoT-SORT (параметры по умолчанию):

Алгоритм	bytetrack	botsort
ID	144.00	130.00
Макс. одновременно	28.00	28.00
Объектов на кадр	3.47	3.64
Средняя длина	28.90	33.60
Медиана	4.00	4.00
Коротких, %	79.90	81.50
Gaps	233.00	291.00
Кадров без объектов	10.00	5.00
FPS	9.70	8.10

Эксперимент	Изменение	ID	Объектов на кадр	Средняя длина	Медиана	Коротких, %	Gaps	FPS
bytetrack_default	по умолчанию	144	3.47	28.9	4	79.9	233	9.7
botsort_default	по умолчанию	130	3.64	33.6	4	81.5	291	8.1
bytetrack_buffer15	{'track_buffer': 15}	148	3.46	28.1	4	79.1	227	12.1
bytetrack_buffer60	{'track_buffer': 60}	142	3.47	29.4	4	78.9	236	11.7
bytetrack_buffer120	{'track_buffer': 120}	137	3.47	30.4	4	79.6	244	11.7
bytetrack_high0.15	{'high_thresh': 0.15, 'new_track_t	118	3.95	40.2	5	67.8	129	11.8
bytetrack_high0.3	{'high_thresh': 0.3, 'new_track_t	87	4.31	59.5	9	59.8	40	11.8
bytetrack_match0.6	{'match_thresh': 0.6}	472	2.32	5.9	1	94.5	224	11.7
bytetrack_match0.95	{'match_thresh': 0.95}	105	5.44	62.2	14	52.4	225	11.7
botsort_buffer60	{'track_buffer': 60}	122	3.66	36.0	4	81.1	306	9.8
botsort_high0.15	{'high_thresh': 0.15, 'new_track_t	108	4.15	46.1	5	73.1	161	9.8
botsort_gmc_none	{'gmc_method': 'none'}	148	3.46	28.1	3	81.1	229	11.7
botsort_prox0.3	{'proximity_thresh': 0.3}	130	3.64	33.6	4	81.5	291	9.7
botsort_prox0.7	{'proximity_thresh': 0.7}	130	3.64	33.6	4	81.5	291	9.7
botsort_reid	{'with_reid': True}	488	7.95	19.6	4	81.6	587	9.6







На видео из 1201 кадра с параметрами по умолчанию ByteTrack создал 144 ID, BoT-SORT 130, средняя длина трека 28,9 и 33,6 кадра, около 80 % треков короче 15 кадров, разрывов 233 и 291. При этом в кадре одновременно не более 8 цифр, окно и подпись, поэтому треки сильно фрагментированы из-за нестабильной уверенности детектора. Лучше всего результат улучшило повышение порогов track_high_thresh и new_track_thresh (ByteTrack при 0,3: 87 ID, 40 разрывов) и match_thresh = 0,95 (105 ID, средняя длина 62,2 кадра), а match_thresh = 0,6 ухудшил его (472 ID). track_buffer и proximity_thresh почти не повлияли, отключение компенсации движения камеры увеличило число ID со 130 до 148, а ReID сильно ухудшил трекинг (488 ID). Главное ограничение остаётся в качестве детекции: даже лучшие конфигурации дали более 80 ID.

Вывод: осуществила обучение нейросетевого детектора для решения задачи обнаружения заданных объектов.